Report

IUM - TWEB

Academic Year 2024/205

UNIVERSITÀ
DI TORINO

Students
Giuseppe Falcone 1050223 giuseppe.falcone@edu.unito.it
Gabriele Vettorello 1014254 gabriele.vettorello@edu.unito.it

Introduction **2**
Task 1. Frontend User Interface To Query and Explore Data **2**
1.1 Solution 2
1.2 Issues 2
1.3 Requirements 2
● HTML pages are required to enable data exploration 2
● JavaScript is required to provide functions that enrich the user experience 2
● CSS is used for consistent styling across the interface 2
● Bootstrap is used to simplify web page styling and structure 2
1.4 Limitations 2
Task 2. Central Orchestration Server (Express.js) **2**
2.1 Solution 3
2.2 Issues 3
2.3 Requirements 3
2.4 Limitations 3
Task 3. Dynamic Data Server (Express.js) and Database (MongoDB) **3**
3.1 Solution 3
3.2 Issues 4
3.3 Requirements 4
3.4 Limitations 4
Task 4. Static Data Server (Spring Boot) & Database (PostgreSQL) **4**
4.1 Solution 4
4.2 Issues 4
4.3 Requirements 4
4.4 Limitations 5
Task 5. Real-Time Chat System **5**
5.1 Solution 5
5.2 Issues 5
5.3 Requirements 5
5.4 Limitations 6
Task 6. Large Scale Data Analysis **6**
6.1 Solution 6
6.2 Issues 6
6.3 Requirements 6
Conclusions **6**
Division of Work **6**
Bibliography **7**

Introduction

This report outlines the development of a comprehensive film data exploration platform. It details the creation of a frontend user interface, a central orchestration server, dynamic and static data servers, and a real-time chat system. The project also includes data analysis to give insights on movies related data. Moreover, it covers the technical solutions implemented, challenges faced, requirements met, and limitations encountered during the development of each platform component.

Task 1. Frontend User Interface To Query and Explore Data

1.1 Solution

The frontend interface is designed to enable users to query and explore film data effectively. It is built using HTML, CSS, JavaScript, Bootstrap, and Handlebars. The interface provides several key page functionalities:

- **Homepage (`index.hbs`)**: This page aims to engage users immediately by presenting random movies categorized by genre, latest releases, and most awarded films.
- **Movies Page (`movies.hbs`)**: This page allows targeted discovery through multi-criteria filtering, enabling users to narrow down large datasets efficiently.
- **Search Functionality**: Users can search for movies by specific categories such as movie title, language, and actor, with an input value providing direct access to information.
- **Single Movie Page (`single-movie-page.hbs`)**: This page consolidates all relevant movie details, including Oscar wins and reviews, with client-side filtering and sorting options for reviews.
- **Actor/Crew Page (`person-page.hbs`)**: This page allows exploration of an individual's career by displaying their filmography and Oscar accolades.

Moreover, all requests are done via Axios calls inside the Javascript files.

1.2 Issues

Difficulties were faced in implementing functionalities to "*load more movies*" and "*apply filters*" in the "movies" page. These issues were resolved by creating a `MovieManager` class with global states and functions to manage asynchronous data updates and filter states.

1.3 Requirements

- HTML pages are required to enable data exploration.
- JavaScript is required to provide functions that enrich the user experience.
- Axios is used to facilitate HTTP requests to the central server.
- CSS is used for consistent styling across the interface.
- Bootstrap is used to simplify web page styling and structure.

1.4 Limitations

- Review filtering/sorting is achieved by storing reviews from the server in an array, which may lead to performance issues if a movie has a large number of reviews.
- Accessibility improvements are needed.

Task 2. Central Orchestration Server (Express.js)

2.1 Solution

The Central Express Server acts as the primary entry point for clients. Its core functions include serving the frontend UI, handling user requests, orchestrating data retrieval from backend servers, and managing the real-time chat system. The server is a Node-Express server located in `solution/central-server`.

- **2.1.1 Handlebars View Engine:** Handlebars is used to render pages dynamically. Templates are organized for reusability and clarity:
    - `central-server/views/layout/layout.hbs`: Provides a consistent page structure.
    - `central-server/views/partials/` (e.g., `navbar.hbs`, `footer.hbs`, `movie-card.hbs`): Contains reusable UI components.
    - `central-server/views/pages/` (e.g., `index.hbs`, `movies.hbs`, `single-movie-page.hbs`): Defines the content for specific views.

- **2.1.2 Routing and Orchestration Logic:** The server uses several routes to handle different parts of the web app:
    - `routes/index.js`: Manages the homepage (`/`), search functionality (`/search`), and the chat (`/chat`).
    - `routes/movies.js`: Handles the movie listing page (`/`), single movie page, and provides an API endpoint to fetch reviews for a movie (`/reviews-by-movie-title`).
    - `routes/people.js`: Serves pages for individual actors (`/actor/:actorName`) and crew members (`/crew/:crewName`).

- **2.1.3 Backend Data Retrieval:** The server uses Axios calls with async/await and Promise.all to facilitate HTTP communications with backend servers and fetch data.
- **2.1.4 API exposure & Documentation:** The server exposes JSON API endpoints that proxy requests to backend servers. Swagger/OpenAPI documentation is available via SwaggerUI at `/api-docs`.
- **2.1.5 Real-Time Chat System:** Achieved using Socket.io.

2.2 Issues

The main issues arose when handling asynchronous calls to retrieve data from the backend servers and aggregate the data.

2.3 Requirements

- The server is built with Express.js.
- It communicates with other servers for database access using Axios.
- It is designed to be fast and delegate database access and data logic to the two backend servers.
- It implements a Real-Time chat system using Socket.io.
- Documentation is provided via Swagger UI.

2.4 Limitations

- Hardcoded URLs for microservices limit deployment flexibility.
- Error handling is currently limited to rendering an error page.
- Scalability may be challenging if the number of microservices increases.

Task 3. Dynamic Data Server (Express.js) and Database (MongoDB)

3.1 Solution

The Dynamic Data Server is a dedicated Express.js microservice providing APIs to access Oscar awards and Rotten Tomato Reviews stored in a MongoDB database.

- **3.1.1 MongoDB Connection and Schemas:** The server connects to MongoDB via Mongoose and defines schemas for two collections: Reviews and Oscars. MongoDB was chosen for storing Reviews data due to its dynamic nature. Centralizing “movie id lacking data” (Oscars and Reviews) into one microservice simplifies orchestration logic in the Central Server
- **3.1.2 API Design and Functionality (Routes & Controllers):** The server exposes RESTful APIs using a controller-service pattern. APIs are divided into:
    - Reviews API: Exposed in `routes/reviews.js`, allowing retrieval of reviews by rate type and movie title.
    - Oscars API: Exposed in `routes/oscars.js`, allowing retrieval of Oscar data for movies or individuals. Controllers call Mongoose model methods directly for asynchronous operations.
- **3.1.3 Documentation:** The server provides API documentation via Swagger/OpenAPI and serves SwaggerUI at `/api-docs`.

3.2 Issues

- Data Consistency: Reliance on `movie_tile`, `film`, and `name` for data retrieval introduced challenges in ensuring accuracy.

- Complex Data Aggregation (`getRatedMoviesByType`): Fetching top-rated movies by type required careful construction for correctness and efficiency.

3.3 Requirements

- The server is implemented in Express.js.
- The server uses Mongoose for MongoDB connection.
- Dynamic data (Reviews) is stored in MongoDB.
- The server provides APIs for the central server to access data.

3.4 Limitations

- Database Population was made using MongoDB Compass; a database population from CSV files would be preferable.
- Limited CRUD Operations: Creating, deleting, or updating reviews or Oscars entries is not currently possible.

Task 4. Static Data Server (Spring Boot) & Database (PostgreSQL)

4.1 Solution

The server is implemented as a Spring Boot application that interfaces with a PostgreSQL database to store and retrieve movie-related static data.

- **4.1.1 Data Initialization and Management:** Upon startup, the server checks the database and populates it if necessary using `@Component DataInitializer`.
- **4.1.2 Database Schema and Entity Relationships:** The database is defined by 10 JPA entities, each representing a table, with `Movie` as the root entity, having `@OneToMany` and `@OneToOne` relationships with the other entities.
- **4.1.3 API Architecture & Functionality:** A `@RestController MovieController` is the main entry point with the base path `/api/movies`. DTO objects are used to structure JSON responses.
- **4.1.4 Service and Repository Layer and Error Handling:** Business logic is in Service classes, interacting with JPA Repositories. `MovieController` implements error handling for `MovieNotFoundException` and `WrongParamException`.
- **4.1.5 Documentation:** The server is documented using Swagger/OpenAPI, serving SwaggerUI at `/api-docs`.

4.2 Issues

- Setting up entities and their relationships was challenging.
- Ensuring the system was fast enough, particularly the search functionality, was difficult.

4.3 Requirements

- The server is a Spring Boot server.
- Static Data is stored in a PostgreSQL database.
- RESTful APIs are provided for the Central Server.
- Documentation is provided.

4.4 Limitations

- Data initialization should be improved, as large CSV sources could lead to long startup and loading times.
- Advanced Query Capabilities: More specialized backend processing would be needed to improve database query capabilities.

Task 5. Real-Time Chat System

5.1 Solution

The Real-Time Chat System is implemented using Socket.io, facilitating bidirectional and event-based communication between clients and the server. This system is an integral part of the Central Orchestration Server.

- **Server-Side (*solution/central-server/socket.io/socket.io.js*):**
  - The server-side logic initializes multiple namespaces (e.g., 'general', 'movies', 'actors', 'countries', 'genres') to segment chat areas.
  - It manages user connections, disconnections, and chat room functionalities within each namespace.
  - Key functionalities include:
    - **Chat Room Management**: Users can join existing chat rooms or create new ones. When a user joins or leaves a room, system messages are broadcast to other users in that room.
    - **Message Handling**: New messages are received from clients, stored in an in-memory chat history for each room, and broadcast to all other clients in the specific chat room.
    - **Chat History**: Upon joining a chat, clients receive the existing message history for that room.
    - **Typing Indicators**: The system broadcasts typing events to show when a user is typing and when they stop.
    - **Chat List Updates**: Clients are sent an updated list of available chat rooms when new chats are created.
    - Chat history is stored in an in-memory object (chats) on the server, initialized with a 'General' chat room.
- **Client-Side (solution/central-server/public/javascripts/chat.js):**
  - The client-side script handles user interface interactions and manages the Socket.io connection.
  - **Username Management**: Users are prompted for a username, which is stored in localStorage for persistence across sessions. Users can change their username.
  - **Namespace and Chat Room Switching**: Users can switch between different namespaces (e.g., 'general', 'movies'), which disconnects from the current namespace and connects to the new one, resetting the chat state. Users can also switch between chat rooms within a namespace or exit the current chat.
  - **Message Display and Sending**: Messages are displayed in the chat area, distinguishing between the current user's messages, other users' messages, and system messages. Users can send new messages, which are emitted to the server along with a timestamp.
  - **UI Updates**: The chat list is dynamically updated. A "Select a chat" message is shown or hidden based on whether a chat is active.
  - **Typing Indicators**: The UI displays when another user is typing.
  - **Sound Notifications**: A chime sound is played for new messages, with an option to mute/unmute.
- **Routing (solution/central-server/routes/index.js):**
  - The Central Server provides a dedicated route (/chat) to render the chat interface page (pages/chat.hbs).

5.2 Issues

No Issues

5.3 Requirements

- The system implements a real-time chat functionality using Socket.io.

5.4 Limitations

- **In-Memory Chat History**: Chat history is stored in memory on the server. This means that if the server restarts, all chat history will be lost. This approach also might not scale well with a very large number of active chat rooms or messages.
- **No Persistent User Accounts**: User identity is managed by a username stored in localStorage on the client-side. There is no server-side user authentication or persistent user profile system.
- **Basic Chat Features**: The system provides core chat functionalities but lacks advanced features such as private messaging between users, message editing/deletion, rich text formatting, or file sharing.

Task 6. Large Scale Data Analysis

6.1 Solution

We analyzed the provided dataset dividing the work into 2 main tasks:

- **Data Cleaning:** After an initial check, it was clear that the dataset, composed of 12 .csv files, needed to be cleaned. This was done in 12 different jupyter notebooks, that clean the data frames and save them in .csv files.
- **Creation of Data Loading module:** The creation of a python module was crucial to simplify the process, not only there are no “path” errors, but also dtypes casting is applied on reading the files.
- **Creation of a merged data frame and storing it into a .parquet file:** for the sake of readability, the data needed for *movies-oscar-analysis.ipynb* was merged into a single dataframe beforehand and saved into a .parquet file to maintain dtypes.
- **Data Analysis and Visualization:** Two different analysis were made:
    - *movies-oscar-analysis.ipynb*: The analysis focuses on finding common trends in movies, in particular in Oscar acclaimed movies, with a focus on non-US movies.
    - actors-analysis.ipynb: The analysis pivots around actors, what influences the career of an actor?

6.2 Issues

No issues found

6.3 Requirements

- Used *pandas* and *numpy* to perform operations on dataframes.
- Used *plotly, matplotlib*, *seaborn*, *pycountry* to analyze and visualize the data.
- Performed a data cleaning stage.

Conclusions

This project successfully implemented a multi-tiered architecture for a film data exploration platform, featuring a frontend interface, a central orchestration server, dynamic and static data servers, and a real-time chat system. Data analysis and visualization were performed to gain insights from the dataset. While some limitations were identified, the core functionalities were effectively delivered.

Division of Work

- **Falcone Giuseppe:** 70% Front-End, 50% Central-Server, 50% Dynamic-Data-Server, 60% Static-Data-Server, 20% Chat-System**,** 50% Data Analysis
- **Vettorello Gabriele:** 30% Front-End, 50% Central-Server, 50% Dynamic-Data-Server, 40% Static-Data-Server, 80% Chat-System**,** 50% Data Analysis

Bibliography

- AI tools: ChatGPT, GitHub Copilot, Google Gemini were used to create the logo, to rephrase documentation and comments, and to generate styling color schemes.